

Trabalho Prático 2 — Parte 1

Pipeline de *Information Retrieval* e *Question Answering*
sobre artigos da Wikipedia

João Moura

Universidade do Minho — Mestrado em Engenharia Informática
Scripting no Processamento de Linguagem Natural (SPLN) — 2025/26

Maio de 2026

Abstract

Este relatório descreve a implementação de uma *pipeline* de *Information Retrieval* (IR) e *Question Answering* (QA) construída sobre um corpus de artigos da Wikipedia. Como tema foram escolhidos os países do mundo, tendo sido construído um corpus de 194 artigos partidos em 8919 *chunks*. O sistema implementa um *retriever* híbrido que combina *BM25* (léxico) e *Sentence-BERT* (semântico) através de *Reciprocal Rank Fusion*, e dois sistemas de QA — um extractivo, com *fine-tuning* de DistilBERT sobre o dataset SQuAD (EM 69.80, F1 78.52), e outro abstrativo, baseado num esquema RAG (*Retrieval-Augmented Generation*) com o modelo *FLAN-T5-base* e uma camada de pós-processamento que filtra respostas espúrias. Os resultados mostram que os dois paradigmas de QA são complementares: o extractivo é mais preciso em factóides; o abstrativo é mais flexível em perguntas relacionais e — graças à camada de filtragem — abstém-se de forma robusta em situações em que não consegue extrair resposta limpa.

1 Introdução

O Trabalho Prático 2 da UC de SPLN tem como objetivo a implementação prática de uma *pipeline* de *Information Retrieval* e *Question Answering*, integrando os conceitos estudados ao longo do semestre: *word embeddings*, *transformers*, *fine-tuning* de modelos pré-treinados e técnicas de *retrieval* léxico e semântico.

Os requisitos do enunciado especificam três tarefas: (a) construção de um corpus com pelo menos 100 documentos sobre um tema à escolha; (b) implementação de um módulo *retriever* capaz de devolver os documentos mais relevantes para uma *query*; (c) implementação de duas estratégias de *Question Answering* — uma extractiva, com *fine-tuning* no SQuAD [4], e uma abstractiva, com *prompting* a um modelo generativo.

Optou-se por construir o corpus a partir de artigos da Wikipedia em inglês sobre os países do mundo. A escolha foi motivada por três fatores: (i) cumprimento confortável do requisito mínimo de documentos (193 Estados-membros da ONU mais alguns observadores); (ii) artigos de qualidade editorial elevada e estrutura relativamente uniforme; (iii) riqueza factual que permite formular perguntas variadas (geográficas, demográficas, históricas, políticas) para avaliação do sistema.

Todo o sistema está implementado num único *Jupyter notebook* (`tp2.ipynb`) com seis secções correspondentes às fases da *pipeline*: corpus, pré-processamento, *retriever*, QA extractivo, QA abstrativo e demonstração.

2 Construção do Corpus

O corpus foi construído a partir da biblioteca `wikipedia-api` (*wrapper* oficial da API REST da Wikipedia). Para cada país da lista foi descarregado o artigo Wikipedia correspondente, com os seguintes cuidados:

- **User-Agent próprio**, em conformidade com a política da Wikipedia, identificando o projeto académico.
- **Resolução de ambiguidades**: alguns títulos de países colidem com outras entidades (por exemplo, “Georgia” refere tanto o país do Cáucaso como o estado dos EUA). Foi criado um mapeamento manual de *overrides* para esses casos.
- **Filtragem de secções navegacionais**: secções como *See also*, *References*, *External links*, *Notes*, *Further reading* e *Bibliography* foram excluídas por não conterem conteúdo factual relevante.
- **Limpeza de texto**: remoção de marcadores de referência ([1], [2], etc.) e normalização de espaçamento.
- **Cache em disco**: cada artigo é guardado num ficheiro `.txt` próprio, permitindo retomar a recolha em caso de interrupção sem repetir pedidos à API.

Estatísticas do corpus bruto

Métrica	Valor
Países na lista	196
Artigos descarregados com sucesso	194
Total de <i>chunks</i> após processamento	8 919
Média de <i>chunks</i> por país	46.0

Table 1: Estatísticas do corpus após recolha.

3 Pré-processamento e *Chunking*

Os artigos da Wikipedia têm tipicamente dezenas de milhares de caracteres — demasiado para passar diretamente a um modelo de QA, que num *transformer* típico está limitado a 512 *tokens* de contexto. Além disso, o *retriever* funciona melhor sobre passagens curtas e tematicamente coerentes do que sobre artigos completos.

A estratégia adotada foi a divisão por **parágrafos naturais** (separados por dupla quebra de linha no texto extraído da Wikipedia), com filtragem de parágrafos com menos de 30 palavras (geralmente cabeçalhos, frases soltas ou *stubs*). Cada *chunk* é guardado num único ficheiro `corpus.json` com a estrutura `{id, country, text}`.

3.1 Limpeza profunda do corpus

Uma primeira iteração do sistema revelou que a limpeza inicial (apenas o padrão numérico `[\d+]`) deixava passar várias variantes de marcadores de citação da Wikipedia que, ao serem passadas ao modelo abstrativo, provocavam respostas espúrias compostas apenas por esses marcadores (Secção 7). Foi então adicionada uma função de limpeza mais abrangente, aplicada por parágrafo após o *split*:

Listing 1: Limpeza profunda aplicada a cada parágrafo.

```
def clean_corpus_text(text):
    # Colchetes: [1], [iv], [a], [4]., etc.
    text = re.sub(r'\[[\w\d]{1,5}\]\.?', '', text)
    # Parenteses com 1-2 dígitos: (4), (10).
    # (nao apanha anos como (1990))
    text = re.sub(r'\(\d{1,2}\)\.?', '', text)
    # Parenteses com 1 letra: (A), (b).
    # (nao apanha siglas como (EU), (UN), (NATO))
```

```

text = re.sub(r'\([A-Za-z]\)\.?', '', text)
text = re.sub(r'\s+', ' ', text)
return text.strip()

def chunk_article(text, min_words=30):
    paragraphs = [p.strip() for p in text.split("\n\n") if p.strip()]
    cleaned = [clean_corpus_text(p) for p in paragraphs]
    return [p for p in cleaned if len(p.split()) >= min_words]

```

A abordagem foi preferida a alternativas (janela deslizante de tamanho fixo, divisão por frase) por duas razões: (i) preserva a coerência semântica do parágrafo original, que tipicamente trata um único tópico; (ii) tem implementação simples e reproduzível, sem hiperparâmetros sensíveis.

Métrica	Valor
Total de <i>chunks</i>	8 919
Média de <i>chunks</i> por país	46.0
Média de palavras por <i>chunk</i>	230
Mediana de palavras por <i>chunk</i>	202
Mínimo / máximo de palavras por <i>chunk</i>	30 / 1 326

Table 2: Estatísticas após *chunking*.

4 Retriever Híbrido

O módulo *retriever* foi implementado em três variantes, com o objetivo de comparar abordagens léxicas, semânticas e híbridas.

4.1 BM25 (léxico)

A primeira abordagem usa **BM25** [7], uma função de pontuação que generaliza o TF-IDF e que continua a ser uma *baseline* forte em *retrieval*. A implementação usa a biblioteca **rank-bm25** (**BM25Okapi**). A tokenização é simples: *lowercase* e *split* por caracteres alfanuméricos (`re.findall(r"\w+", text.lower())`). Não foi feita remoção de *stopwords* porque o próprio BM25 ajusta o peso de termos frequentes através do IDF.

4.2 Sentence-BERT (semântico)

A segunda abordagem usa *embeddings* densos calculados com **Sentence-BERT** [3]. O modelo escolhido foi o **multi-qa-MiniLM-L6-cos-v1**, especificamente treinado para *retrieval* em contexto de QA, com 384 dimensões e tamanho reduzido (cerca de 80 MB).

Os *embeddings* de todos os *chunks* são pré-calculados uma única vez e guardados em **embeddings.npy**. Para uma *query*, calcula-se o *embedding* e ordenam-se os *chunks* por similaridade cosseno.

4.3 Híbrido — *Reciprocal Rank Fusion*

A terceira abordagem combina as *rankings* dos dois *retrievers* através de **Reciprocal Rank Fusion** (RRF) [8], definida por:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \quad (1)$$

onde $R = \{\text{BM25}, \text{Sentence-BERT}\}$, $\text{rank}_r(d)$ é a posição do documento d na *ranking* do *retriever* r , e $k = 60$ é uma constante de suavização típica da literatura. A vantagem do RRF é combinar

rankings em vez de *scores* brutos, evitando o problema das escalas diferentes (BM25 produz *scores* em $R_{\geq 0}$; *cosine* entre -1 e 1).

4.4 Análise comparativa

Para avaliar qualitativamente os três *retrievers*, foram usadas três *queries* representativas de padrões distintos de pergunta:

- **Q1.** *What is the capital of Portugal?* — factóide direto.
- **Q2.** *Which countries border Brazil?* — pergunta relacional.
- **Q3.** *When did Germany reunify?* — envolve paráfrase.

Query	BM25 (top-1)	Semantic (top-1)	Híbrido (top-1)
Q1: Capital Portugal	<i>chunk</i> sobre Neanderthals (ocorrências literais de “Portugal” mas irrelevante)	<i>chunk</i> com “Lisbon as its capital” — correto	<i>chunk</i> com “Lisbon as its capital” — correto
Q2: Fronteiras Brasil	Uruguai (artigo sobre disputa territorial com o Brasil)	Brasil (<i>chunk</i> com lista de países vizinhos) — correto	Uruguai — <i>ranking</i> arrastada pela concordância
Q3: Reunificação Alemanha	Camarões, Luxemburgo, Marshall Is. (mencionam “Germany” em contextos errados)	Três <i>chunks</i> sobre a Alemanha, incluindo o ato de reunificação — correto	<i>chunk</i> sobre capitulação Nazi (início da história da reunificação)

Table 3: Resultados dos três *retrievers* para as *queries* de teste.

Conclusões da análise:

- O **BM25** é o melhor quando a pergunta usa as mesmas palavras do texto, mas é facilmente enganado por frequência elevada de termos em contextos errados.
- O **retriever semântico** é claramente superior em *queries* com paráfrases (Q3) e relacionais (Q2).
- O **híbrido (RRF)** é robusto na maior parte dos casos, mas em Q2 produziu um resultado pior que o semântico isolado — o RRF favorece o consenso entre as duas *rankings* e isto pode prejudicar quando uma delas está fortemente correta e a outra fortemente errada.

5 Question Answering Extractivo

A primeira estratégia de QA é *extractiva*: o modelo recebe uma pergunta e um contexto, e devolve uma *span* (sequência contígua de *tokens*) do contexto que constitui a resposta. Esta abordagem é típica em modelos baseados em BERT [1].

5.1 Modelo e *Fine-Tuning*

Foi escolhido o **distilbert-base-uncased** [2], uma versão destilada (40% mais pequena, 60% mais rápida) do BERT que preserva cerca de 97% do desempenho em tarefas *downstream*. O *fine-tuning* foi efetuado no SQuAD v1.1 [4], o dataset de referência para QA extractivo.

Por restrições de tempo de computação, foi usado um subconjunto de 20 000 exemplos de treino e 500 de validação. O treino correu em cerca de 30 min numa GPU T4 do Google Colab, com a configuração detalhada na Tabela 4.

Hiperparâmetro	Valor
Modelo base	distilbert-base-uncased
Tamanho máximo de sequência	384
<i>Stride</i> (overflow)	128
<i>Learning rate</i>	3×10^{-5}
Épocas	2
<i>Batch size</i>	16
<i>Weight decay</i>	0.01
Precisão	FP16 (em GPU)

Table 4: Hiperparâmetros do *fine-tuning*.

5.2 Resultados

A avaliação foi efetuada num subconjunto de 500 exemplos da partição *validation* do SQuAD, com as métricas oficiais Exact Match (EM) e F1.

Métrica	Valor
Exact Match (EM)	69.80
F1 score	78.52

Table 5: Resultados no SQuAD validation (500 exemplos).

A literatura reporta para DistilBERT com *fine-tuning* completo no SQuAD valores na ordem de $EM \approx 77$ e $F1 \approx 85$. Os valores obtidos (EM 69.80, F1 78.52) ficam cerca de 7 pontos abaixo desses, o que é compatível com o uso de um subconjunto de cerca de 23% do total de exemplos de treino e apenas 2 épocas. O resultado é satisfatório e demonstra que o *fine-tuning* convergiu corretamente.

5.3 Integração com o *retriever*

Para a inferência em *queries* reais, o sistema combina o *retriever* híbrido com o modelo extractivo: recupera os 3 *chunks* mais relevantes, executa o modelo QA sobre cada um, e escolhe a resposta com maior *score* de confiança. Esta estratégia permite recuperar quando o *retriever* não devolve o *chunk* ideal em primeiro lugar.

6 Question Answering Abstrativo (RAG)

A segunda estratégia segue o paradigma *Retrieval-Augmented Generation* (RAG) [5]: o *retriever* fornece o contexto a um modelo generativo, que produz uma resposta em linguagem natural, podendo parafrasear, sintetizar ou recusar responder.

6.1 Modelo

Foi usado o `google/flan-t5-base` [6], uma variante *instruction-tuned* do T5 com cerca de 250 M parâmetros. O FLAN-T5 foi treinado em mais de 1800 tarefas formuladas como instruções, o que lhe confere capacidade *zero-shot* robusta para tarefas de QA quando recebe um *prompt* bem estruturado.

6.2 Prompt usado

A construção do *prompt* segue um padrão típico de RAG. Após experimentação empírica (descrita na Secção 7), a versão final usa prosa natural para identificar a origem de cada *chunk*, evitando estruturas com colchetes que o modelo pequeno tendia a imitar nas respostas:

Listing 2: Estrutura final do prompt RAG.

```
Read the context below and answer the question concisely.

Context:
Article about Portugal: <chunk text>

Article about Brazil: <chunk text>

Article about Germany: <chunk text>

Question: <user query>
Answer:
```

A geração foi configurada com *beam search* (4 *beams*) e limite de 80 *tokens* na resposta. O limite de *input* foi fixado em 1024 *tokens*.

6.3 Camada de filtragem (*safety net*)

Mesmo com o corpus limpo e o *prompt* bem estruturado, o FLAN-T5-*base* ocasionalmente gera respostas que são apenas marcadores curtos sem palavras significativas (*eco* de padrões de citação que viu no pré-treino sobre Wikipedia). Para evitar que estas respostas espúrias se confundam com respostas reais, foi adicionada uma camada de pós-processamento que detecta este caso e substitui a resposta por um aviso explícito:

Listing 3: Camada de filtragem aplicada à resposta gerada.

```
answer = t5_tokenizer.decode(outputs[0],
                             skip_special_tokens=True).strip()

has_real_word = bool(re.search(r'[a-zA-Z]{4,}', answer))
is_short_junk = len(answer) <= 12
if not has_real_word and is_short_junk:
    answer = "(no clean answer extracted)"
```

A heurística é simples e transparente: uma resposta válida deve conter pelo menos uma palavra com 4 ou mais letras consecutivas; respostas muito curtas (≤ 12 caracteres) sem nenhuma palavra real são consideradas ruído. Esta camada não tenta corrigir o modelo — limita-se a tornar visível ao utilizador que o sistema não conseguiu extrair uma resposta limpa, o que é claramente preferível a apresentar marcadores como “(D).” ou “[3]. [4].” como se fossem respostas.

7 Demonstração e Análise Crítica dos Sistemas

Para avaliação qualitativa dos dois sistemas, foi construído um conjunto de 8 *queries* cobrindo padrões de pergunta distintos. A Tabela 6 apresenta os resultados finais, obtidos depois da limpeza profunda do corpus e da camada de filtragem.

7.1 Iteração de engenharia: como chegámos a estes resultados

Os resultados acima são o produto de várias iterações sobre uma primeira versão do sistema que apresentava falhas claras. Vale a pena documentar este processo porque é parte integrante do trabalho.

Query	Extractivo	Abstrativo	Categoria
What is the capital of Portugal?	Lisbon ✓	(no clean answer)	Factoide
What is the official language of Italy?	Italian ✓	(no clean answer)	Factoide
What is the population of Iceland?	two million ✗	(no clean answer)	Numérico
Which countries border Brazil?	Uruguay (parcial)	Uruguay to the south ✓	Relacional
When did India gain independence?	1947 ✓	unanswerable ✓	Histórico
When did Germany reunify?	World War II ✗	(no clean answer)	Paráfrase
What currency is used in France?	the euro ✓	franc. ✗	Paráfrase
Who won the 2026 FIFA World Cup?	Qatar ✗	(no clean answer) ✓	Armadilha

Table 6: Comparação extractivo vs. abstrativo nas *queries* de demonstração. As células marcadas ✗ são respostas erradas; (no clean answer) na última linha é uma abstenção correta perante a pergunta-armadilha.

Primeira iteração: artefactos do corpus. A versão inicial produzia respostas como (D)., (4)., [iv] e [3]. [4]. no modelo abstrativo — todas elas marcadores de citação que sobreviveram à limpeza inicial do corpus. A limpeza original (`re.sub(r"[\d+]", "", text)`) só capturava o caso mais comum, mas a Wikipedia também usa numerais romanos entre colchetes ([iv]), letras únicas ([a]), parênteses redondos ((4)) e variações com pontuação adjacente ([4].). Estes padrões alimentavam o modelo, que pequeno como é (250 M parâmetros), os reproduzia.

Segunda iteração: o prompt como template a imitar. Uma observação intermédia surpreendente: quando o *prompt* prefixava cada *chunk* com [Country] (estrutura típica em RAG), o modelo passava a gerar respostas em formato de colchetes também — [B]. para a pergunta sobre o Brasil, [Japanese] para a pergunta sobre o Japão. O modelo aprendia *em tempo real*, dentro do próprio *prompt*, que “as coisas importantes estão entre colchetes”, e mimetizava esse formato. A solução foi substituir o prefixo por prosa natural (“Article about Portugal:”).

Terceira iteração: limpeza profunda do corpus. A função `clean_corpus_text` (Secção 3.1) foi aplicada por parágrafo após o *split*, capturando exaustivamente as variantes de marcadores. Este passo eliminou a maioria dos artefactos.

Quarta iteração: camada de filtragem. Mesmo com o corpus limpo, observou-se que o FLAN-T5-*base* continuava a gerar ocasionalmente respostas-marcador (“eco do pré-treino”). A camada de filtragem (Secção 6.3) intercepta estes casos e substitui-os por (no clean answer extracted), tornando o comportamento honesto e legível.

7.2 Análise dos resultados finais

Sucessos. O *retriever* híbrido cumpriu o seu papel: identificou corretamente os *chunks* relevantes em factoides como “Lisbon” (capital de Portugal), “Italian” (língua de Itália), “the euro” (moeda de França) e “1947” (independência da Índia, vinda do contexto do Paquistão). O modelo abstrativo brilhou nas perguntas em que o extractivo é estruturalmente limitado: “Uruguay to the south” para fronteiras do Brasil — uma resposta em linguagem natural que o extractivo,

devolvendo apenas uma *span*, não conseguiria formular completamente. E na pergunta sobre a Índia, o modelo abstrativo respondeu **unanswerable** quando o *retriever* passou contextos tematicamente próximos mas insuficientes (Paquistão, Butão) — um comportamento emergente de auto-deteção da incapacidade de responder.

Falhas residuais do extractivo. Três casos ilustram limitações reais:

- **Inexatidão numérica:** “two million” para a população da Islândia (valor real $\approx 376\,000$). O modelo apanhou um número de outro contexto.
- **Paráfrase:** “World War II” para a reunificação da Alemanha. O modelo associou o tema “reunificação alemã” ao contexto histórico mais saliente.
- **Pergunta-armadilha:** “Qatar” para o vencedor do Mundial de 2026 — mas Qatar foi sede em 2022, e o Mundial 2026 ainda não tinha ocorrido à data deste trabalho. O modelo extractivo não tem mecanismo de abstenção e devolveu o país com maior *score*.

Comportamento honesto do abstrativo. Após a limpeza do corpus e a camada de filtragem, o modelo abstrativo passou a comportar-se de forma muito mais transparente. Nas perguntas em que não consegue extrair uma resposta limpa, devolve (**no clean answer extracted**) — incluindo na pergunta-armadilha (Mundial 2026), que se torna assim uma *abstenção correta por defesa engenharial*. Esta é a diferença fundamental face à primeira iteração: o sistema agora distingue claramente entre “isto é uma resposta” e “não consegui responder”.

7.3 Lições aprendidas

1. A **qualidade do *retriever*** continua a ser o gargalo: chunks errados produzem respostas erradas. A limpeza do corpus melhora a relevância dos chunks indexados.
2. A **limpeza do corpus** tem de ser exaustiva. Marcadores de citação que sobrevivem à limpeza inicial não são apenas “ruído visual” — são tokens que modelos pequenos aprendem a gerar.
3. Modelos **abstrativos pequenos são sensíveis ao formato do prompt**. Estruturas como [Country] podem ser interpretadas como template a imitar.
4. A **abstenção honesta** em modelos pequenos pode ser implementada por defesa engenharial (camada de filtragem), em vez de depender exclusivamente da capacidade emergente do modelo. É um caso interessante em que uma heurística simples produz comportamento mais fiável do que confiar no modelo isolado.

8 Conclusões

Foi implementada uma *pipeline* completa de IR e QA sobre um corpus de 8 919 *chunks* extraídos de 194 artigos Wikipedia sobre os países do mundo. O *retriever* híbrido combina BM25 e Sentence-BERT através de RRF, e os dois sistemas de QA cobrem as duas estratégias clássicas (extractiva via DistilBERT *fine-tuned* no SQuAD com EM 69.80/F1 78.52; abstractiva via RAG com FLAN-T5-*base* e camada de filtragem).

A análise comparativa permitiu três conclusões principais:

1. O ***retriever* semântico** é claramente superior ao léxico em *queries* com paráfrases, mas o léxico mantém-se útil para entidades nomeadas raras. O híbrido captura o melhor dos dois *na maioria* dos casos.
2. Os dois sistemas de **QA** são complementares: o extractivo é mais preciso em factoides simples mas estruturalmente incapaz de respostas relacionais; o abstrativo é mais flexível em paráfrases e em perguntas relacionais, e, com a camada de filtragem, abstém-se de forma robusta perante perguntas sem resposta no corpus.

3. A **engenharia em volta dos modelos** (limpeza do corpus, formato do *prompt*, camada de filtragem) tem um impacto comparável a aumentar o tamanho do modelo. Para um sistema produtivo, esta camada é frequentemente o melhor investimento.

Trabalho Futuro

- **Treino completo do extractivo** no SQuAD (88 000 exemplos, 3 épocas) deve elevar EM e F1 para a faixa de referência ($\approx 77/85$).
- **Modelos abstrativos maiores** (FLAN-T5-*large*, FLAN-T5-XL) dariam respostas de qualidade superior em perguntas multi-*hop* e poderiam responder a perguntas onde o sistema atual abstém-se.
- **Avaliação quantitativa *end-to-end*** sobre um conjunto de pares pergunta-resposta construído sobre os artigos dos países, permitindo medir *accuracy* formalmente em vez da análise qualitativa aqui feita.
- **Substituir a heurística de filtragem por classificador aprendido**, capaz de detectar respostas-lixo com base em sinais mais ricos (perplexidade da geração, divergência com o contexto, etc.).

References

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *NAACL*, 2019.
- [2] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108, 2019.
- [3] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *EMNLP*, 2019.
- [4] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In *EMNLP*, 2016.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*, 2020.
- [6] Hyung Won Chung, Le Hou, Shayne Longpre, et al. Scaling Instruction-Finetuned Language Models. arXiv:2210.11416, 2022.
- [7] Stephen Robertson and Hugo Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [8] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods. In *SIGIR*, 2009.